先给一句话总结

HTTP/2.0 相比 HTTP/1.x：用同一条连接并发多请求、用二进制帧传输、压缩头部、支持服务器推送，极大减少“连接开销 + 队头阻塞 + 重复头部”的浪费。

---

1. 连接与并发：多路复用 vs 多连接

- HTTP/1.0
  - 一次请求 → 一条 TCP 连接，用完就断（短连接）
- HTTP/1.1
  - 支持 **Keep-Alive**：一条连接可以发多个请求
  - 但**同一时刻一个连接上基本只能按顺序处理请求**
  - 浏览器为了并发，只能对同一域名开很多 TCP 连接（比如 6 条）
- HTTP/2.0
  - **多路复用（Multiplexing）**：
    - **一条 TCP 连接上同时跑很多请求**，每个请求有自己的 `stream id`
    - 数据被拆成帧交织发送，不会互相阻塞（应用层不队头阻塞）
  - 结果：
    - 减少 TCP 连接数量
    - 降低握手成本和 TCP 慢启动的影响
    - 页面资源加载更快、更稳定

---

2. 文本协议 vs 二进制帧

- HTTP/1.x
  - 文本协议，报文是“可读字符串”，解析成本高，不利于扩展
- HTTP/2.0
  - 二进制分帧（Binary Framing）
    - 所有请求/响应都被拆成一个个“二进制帧”
    - 不同类型：HEADERS 帧、DATA 帧等
    - 易于机器解析和扩展（例如优先级、流量控制等）

---

3. 头部压缩：HPACK

- HTTP/1.x
  - 每次请求都带完整的请求头（Host、Cookie、User-Agent…）
  - 很多字段在同一个连接的不同请求中基本不变 → 大量冗余
- HTTP/2.0
  - 使用 **HPACK 头部压缩**：

        - 维护一张“头字段表”，利用索引 + Huffman 编码来压缩头部
        - 在很多小资源请求场景（CSS/JS/图片）能显著减小体积

---

4. 服务器推送（Server Push）

- HTTP/1.x
    - 客户端只能“要什么，服务器给什么”
- HTTP/2.0
    - 支持 **Server Push**：
        - 客户端访问 `index.html` 时，服务端可以**主动**顺带推送 `index.css` 、 `index.js`
        - 理论上能减少一次 RTT（请求-响应往返）
    - 实际上浏览器/中间层的支持与滥用问题较多，**很多团队现在倾向少用或干脆关掉**（尤其是配合 HTTP/3、`preload` 等新机制）

---

5. 优先级与流量控制

- HTTP/1.x
    - 基本没有细粒度的流优先级控制
- HTTP/2.0
    - 每个流可以设定优先级，浏览器可以告诉服务器“首屏核心资源先来”
    - 有更细粒度的**流量控制**，避免某个大请求独占带宽

---

6. 与 TCP、HTTPS 的关系

- **HTTP/2.0 仍然跑在 TCP 之上**（主流实现）
- 主流浏览器会要求 HTTP/2 必须基于 TLS（即 HTTPS）
    - 所以实际使用中几乎都是：**HTTPS（TLS） + HTTP/2 + TCP**
- HTTP/2 解决的是 **应用层的队头阻塞 & 连接滥用问题**，
  TCP 层本身的队头阻塞问题在 **HTTP/3（基于 UDP + QUIC）** 中才有更好的解决。

---

7. 简表对比

特性	HTTP/1.0	HTTP/1.1	HTTP/2.0
连接方式	短连接	长连接（Keep-Alive）	单连接多路复用
并发	无	需多 TCP 连接实现	单 TCP 连接内多 stream 并发
报文格式	文本	文本	二进制帧
头部压缩	无	无	有（HPACK）
服务器推送	无	无	有（Server Push）
优先级 / 流控	弱	弱	强，支持流优先级和精细流量控制